What is sub-array in Java ?
----
* A sub array is the **contiguous** part of the array which we can write in various ways.

* The elements of the array must be **contiguous**.

Example of sub array :
----

int []arr = {1,2,3,4,5};

Range of index	Sub-array
0 - 0	**1**
0 - 1	1 2
0 - 2	1 2 3
0 - 3	1 2 3 4
0 - 4	1 2 3 4 5
1 - 1	**2**
1 - 2	2 3
1 - 3	2 3 4
1 - 4	2 3 4 5
2 - 2	**3**
2 - 3	3 4
2 - 4	3 4 5
3 - 3	**4**
3 - 4	4 5
4 - 4	**5**

i,j
0 1 2 3 4
1, 2, 3, 4, 5

WAP to print the sub-array :

```
 package com.ravi.one_diemensional_basic;

 public class ArrayDemo24 {

     public static void main(String[] args)
     {
         int []arr = {1,2,3,4,5};

         for(int i=0; i<arr.length; i++)
         {
             for(int j=i; j<arr.length; j++)
             {
                 for(int k=i; k<=j; k++)
                 {
                     IO.print(arr[k]+" ");
                 }
                 IO.println();
             }
         }

     }

 }
```

Q25) Find The Maximum Subarray Sum (Leetcode 53) [Kadane's Algorithm]
[You are given an integer array nums. Your task is to find a contiguous subarray (containing at least one element) that has the largest sum and return that sum.]

Example : -1, 3, 4, -2

```
Example : -1, 3, 4, -2

Subarray                sum =-2      maxSum = 7     [if(sum > maxSum), maxSum=sum]
  -1                     -1
  -1 3                    2
  -1 3 4                  6
  -1 3 4 -2               4
   3                      3
   3 4                    7
   3 4 -2                 5
   4                      4
   4 -2                   2
   -2                    -2
```

* In order to find the maximum sub-array sum, We need 3 nested loops so, time complexity is not good.

* To provide best time complexity (only one loop) to solve the problem, We should use **Kadane's Algorithm**

****Kadane's Algorithm**

```
5 6 -3 7 -13  8  -2  5  -6  7 -11  3 10  -10  -6 -10  7  2


sub-array                                                         sum=16    maxSum=16
5                                                                    5
5+6                                                                  11
5+6-3                                                                8
5+6-3+7                                                              15
5+6-3+7-13                                                           2
5+6-3+7-13 + 8                                                       10
5+6-3+7-13 + 8 -2                                                    8
5+6-3+7-13 + 8 -2+5                                                  13
5+6-3+7-13 + 8 -2+5 -6                                               7
5+6-3+7-13 + 8 -2+5 -6 + 7                                           14
5+6-3+7-13 + 8 -2+5 -6 + 7 -11                                       3
5+6-3+7-13 + 8 -2+5 -6 + 7 - 11 + 3                                  6
5+6-3+7-13 + 8 -2+5 -6 + 7 - 11 + 3 + 10                             16
5+6-3+7-13 + 8 -2+5 -6 + 7 - 11 + 3 + 10  - 10                       6
5+6-3+7-13 + 8 -2+5 -6 + 7 - 11 + 3 + 10  - 10  -6                   0
5+6-3+7-13 + 8 -2+5 -6 + 7 - 11 + 3 + 10  - 10  -6 -10              -10
7                                                                    7
7 + 2                                                                9
```

Note : Add the array elements to sum till sum >=0, If sum is negative then start own business

//Program :

```
 package com.ravi.one_diemensional_basic;

 public class ArrayDemo25
 {
     public static void main(String[] args)
     {
         //int []nums = {-1, 3, 4, -2};


         //int []nums = {-2,1,-3,4,-1,2,1,-5,4};

         int []nums = {5,  6,  -3,  7,  -13,   8,   -2,   5,   -6,   7,  -11,   3,  10,   -10,   -6,
 -10,  7,  2};

         int maxSum = maxSubArray(nums);
         IO.println("Maximum length of subarray sum is :"+maxSum);
     }


     public static int maxSubArray(int[] arr)
     {
         int sum = 0;
         int maxSum = 0;

         for(int i=0; i<arr.length; i++)
         {
             if(sum >=0)
             {
                 sum = sum + arr[i];
             }
             else
             {
                 sum = arr[i];
             }

             if(sum > maxSum)
             {
                 maxSum = sum;
             }
         }

          return maxSum;

     }

 }
```

What is 2D Array ?

A two dimensional array is an array of arrays.

It stores data in **rows and columns** (like a table or matrix).

Example : 3 rows & 3 columns

1 2 3
4 5 6
7 8 9